

Vint Cerf on Digital Preservation at AAUP 2015

Cite as: Martin Paul Eve, "Vint Cerf on Digital Preservation at AAUP 2015", *eve.gd: Martin Paul Eve*, June 19, 2015, <https://doi.org/10.59348/6nsg9-qk387>.



Vint Cerf is one of the few people in the world who can viably use the phrase “my internet” in a talk and it be true. Tim Berners-Lee developed the Hypertext Transfer Protocol (HTTP) which underlies the transfer of hyperlinked documents on the World Wide Web. This is an Application protocol, meaning that it sits at the highest level of the internet. In the [OSI Model](#), Cerf and his colleagues developed the TCP/IP protocols which are the Network and Transport layers that sit as the fundamental backbone of how packet switched networks communicate. “My internet” indeed.

Cerf was speaking today at the [Association of American University Presses](#) conference and his topic was digital preservation. Cerf pointed out that it's all very well to preserve bits on digital storage media but that, unless we have ways to interpret those bits, they remain just a sequence of encoded numbers. In other words, it is vital that we preserve executable code (including hardware emulation). He pointed to the great [Olive project](#) at CMU as an example.

It was great to hear Cerf speak; it was a rich and interesting talk. I can't record everything he said and don't really want to, but I did get to ask him a question, which I'll record here for posterity and my own records: might we not develop a metadata standard that describes *how* to interpret the bits, rather than preserving software and hardware? (I had in mind thinking about open standards here: if we can tell the future how to interpret the bits, they could write their own implementations).

Cerf's reply was great and, briefly distilled, consisted of two parts (any errors here are in my transcription and interpretation, not in the original):

1. It would probably take more space to store the description than to store a working implementation of the software. (I might also add, myself: in what form do we store the description so that it can be accessed and interpreted?)
2. We may need to preserve *errors*.

That latter point hadn't really occurred to me. Say, in the present, we use a piece of software to interpret a series of bits and it has a mistake in it. If you publish the perfect reference implementation description and somewhat codes new software to handle it, then perhaps it will not "accurately" recover the file because it does not include the errors in the original software.